

SIH-26

AI-Enabled Real-Time Digital Twin System

**for Health Monitoring, Fault Prediction and Mission Reliability
Enhancement of Aero Piston Engines used in MALE UAVs**

Complete Technical, Architectural & Presentation Reference

Organization: DRDO | Department of Defence Production / IDEX

Category: Software | Smart India Hackathon

Repository: github.com/VIKASHL25/SIH-26

Table of Contents

1. Executive Summary
2. The Problem Statement — What DRDO Actually Asked For
3. Understanding the Domain: Physics & Maths of an Aero Piston Engine
4. Our Approach — How We Turned the Problem Statement Into a System
5. The AI/ML Layer — Every Model, Every Feature, Explained
6. Complete System Architecture
7. The Security Layer — Full Explanation
8. File-by-File Codebase Walkthrough
9. Edge AI Benchmark — Onboard vs. Ground Station Deployment
10. Federated Learning Proof-of-Concept
11. Additional Recommendations
12. How to Present This in the Demo
13. Winning Strategies

1. Executive Summary

This document is a complete technical and strategic reference for the MALE UAV Aero Piston Engine Digital Twin project, built for Smart India Hackathon 2026 problem statement SIH-26, issued by DRDO under the Department of Defence Production / IDEX. It exists so that every team member — including someone who has not personally written the ML models, the CAN telemetry layer, or the microservices — can understand, explain, and defend the project end-to-end in front of judges.

The document covers, in order: the original problem statement and what it actually demands; the underlying physics and mathematics of an aero piston engine that the system is built to understand; how the team's engineering choices map onto each requirement; a full technical walkthrough of every model, every feature, and why it was chosen; the complete system architecture and data flow; a file-by-file explanation of the codebase with the core logic reproduced; the security architecture; and finally, a practical guide to presenting the system in a live demo along with concrete strategies to maximise the score.

Read this once end-to-end before the demo. It is written so that even a teammate with zero exposure to the ML/backend code can explain what the system does, why it does it that way, and what tradeoffs were made.

2. The Problem Statement — What DRDO Actually Asked For

2.1 Title

AI-Enabled Real-Time Digital Twin System for Health Monitoring, Fault Prediction and Mission Reliability Enhancement of Aero Piston Engines used in MALE UAVs.

2.2 Background — Why This Problem Exists

Medium Altitude Long Endurance (MALE) UAVs fly long-duration Intelligence, Surveillance, and Reconnaissance (ISR) missions, maritime surveillance, communication relay, and strategic defence sorties. Because these are long, unmanned, and often flown over contested or remote terrain, the single point of failure that ends a mission is usually the propulsion system: if the piston engine fails mid-flight, the result is a mission abort, loss of an expensive strategic asset, or an unsafe forced recovery.

The core complaint DRDO makes about existing UAV engine monitoring is that it is **threshold-based and reactive** — **a warning light turns on only after a sensor has already crossed a hard-coded danger line**. By then the fault has already happened. There is no forward-looking capability to estimate Remaining Useful Life (RUL), no trend detection for slow degradation, and no way to simulate how the engine would behave under a specific future mission profile (e.g., a hot-weather high-altitude sortie) before flying it.

The task is to replace this reactive, threshold-based approach with a Digital Twin — a continuously updated virtual replica of the physical engine, built from real-time sensor data, physics-based models, and AI/ML — that can predict problems before they happen instead of just reporting them after.

2.3 What the System Must Be Able to Do

- Real-time engine parameter visualization
- Monitoring of engine health indicators
- Detection of abnormal operating conditions
- Predicting probable failures before they occur
- Estimating degradation trends and Remaining Useful Life (RUL)
- Simulating engine behaviour under different mission profiles and environmental conditions
- Supporting post-flight analysis and mission replay

2.4 The Six Expected Sub-Systems (A–F)

DRDO's expected solution is broken into six explicit blocks. Everything the team builds should be traceable to one of these.

A. Digital Twin Core Framework

- A virtual engine model that stays synchronized with live engine data
- Modular architecture so the system can scale later
- Real-time data ingestion capability

B. Health Monitoring System

Must continuously track these specific engine parameters and turn them into health indices:

- RPM
- Cylinder Head Temperature (CHT)

- Exhaust Gas Temperature (EGT)
- Oil Pressure & Oil Temperature
- Fuel flow
- Vibration signatures
- Battery / Alternator health
- Injection timing parameters

C. Fault Detection & Predictive Analytics

Must move from threshold alarms to intelligent, predictive diagnosis of:

- Misfire conditions
- Injector abnormalities
- Cooling degradation
- Lubrication issues
- Sensor drift / failure
- Combustion instability
- Overheating trends
- Abnormal vibration patterns

D. AI/ML Layer

- Anomaly detection algorithms
- Remaining Useful Life (RUL) estimation
- Trend analysis
- Predictive maintenance recommendations

E. Simulation & Replay Capability

- Replay of historical mission data
- Environmental condition simulation
- Engine behaviour simulation for: high altitude, endurance missions, hot-weather operation, rapid throttle transitions

F. Visualization Dashboard

- Real-time engine health status
- Fault alerts
- Engine efficiency trends
- Maintenance advisory
- Mission-wise health reports

2.5 Deliverables Explicitly Requested by DRDO

- A functional prototype / software demonstrator
- Digital twin architecture design
- Engine simulation model
- AI/ML-based anomaly detection module
- Visualization dashboard

- Demonstration using simulated or real engine datasets
- Technical documentation and deployment roadmap

2.6 Desired Innovation Areas (bonus points, not mandatory)

- Physics-informed AI
- Edge AI for UAV applications
- Lightweight onboard analytics
- Hybrid thermodynamic + data-driven models
- Federated learning approaches
- Explainable AI for fault diagnosis
- Secure telemetry architecture
- Autonomous maintenance advisory systems

Notice how many of these 'desired innovation' items map directly onto things this project already has: physics-informed AI (the feature engine), Explainable AI (SHAP + counterfactual sensitivity), secure telemetry (the 5-layer security model), Edge AI (the benchmark study), and Federated Learning (the FedAvg proof-of-concept). Very few competing teams will have touched more than one or two of these.

3. Understanding the Domain: Physics & Maths of an Aero Piston Engine

Before any of the code makes sense, it helps to understand what a MALE UAV piston engine actually is, what its sensors are measuring, and why those specific measurements are the ones that predict failure. This section is a plain-language physics primer — enough to hold your own if a judge asks 'why did you pick these features?' or 'what does EGT residual actually mean physically?'

3.1 What kind of engine is this?

A MALE UAV like the TAPAS-BH-201 class typically uses a small, turbocharged, four-stroke internal combustion (IC) piston engine rather than a jet turbine — piston engines are far more fuel-efficient at the lower speeds and altitudes MALE UAVs cruise at, which is exactly what long endurance requires. The tradeoff is that piston engines have many more moving mechanical parts (pistons, valves, injectors, bearings) that wear down and can fail in more varied ways than a turbine.

A four-stroke engine repeats a cycle of four piston movements: intake (air/fuel mixture drawn in), compression (mixture compressed), power (ignition drives the piston down), and exhaust (burnt gases expelled). Every sensor this project reads is measuring some consequence of this cycle happening thousands of times per minute.

3.2 The Core Sensor Parameters and What They Physically Mean

RPM (Revolutions Per Minute)

How fast the crankshaft is spinning. It is the base 'clock speed' of the engine — almost every other parameter (temperature, torque, fuel flow) is expected to scale with RPM in a predictable way. A sudden RPM drop under constant throttle is a classic misfire signature.

CHT — Cylinder Head Temperature

The temperature at the top of the combustion chamber, where cooling fins/airflow remove heat from the cylinder. CHT rises with power output but should stay within a physically expected band for a given RPM/throttle/altitude combination. A CHT that is higher than expected for the current operating point is one of the earliest, most reliable signs of an overheating or cooling problem — which is exactly why the system computes a 'CHT residual' (measured minus expected) rather than just watching the raw temperature.

EGT — Exhaust Gas Temperature

The temperature of gases leaving the cylinder after combustion. EGT is extremely sensitive to the air-fuel mixture: too lean (not enough fuel) burns hotter and pushes EGT up; a misfire drops EGT abruptly for that cylinder's cycle. EGT is one of the fastest-reacting signals to combustion problems, which is why it's central to the fault classifier.

Oil Pressure & Oil Temperature

Oil is the engine's lubrication and secondary cooling system for moving metal parts (bearings, camshaft, pistons). Falling oil pressure combined with rising oil temperature is the classic signature of lubrication breakdown — a slow, dangerous fault type because it can silently damage bearings over many flight hours before a catastrophic seizure.

Vibration RMS (Root Mean Square)

A vibration sensor (accelerometer) samples mechanical oscillation at high frequency; RMS is the standard way to compress that noisy waveform into a single 'how rough is the engine running right now' number: $RMS = \sqrt{(1/N) \times \sum x_i^2}$ over N samples. Rising vibration RMS over time is a strong indicator of mechanical wear — loose components, bearing wear, or piston/valve imbalance.

Fuel Flow & Air Mass Flow

Fuel flow (kg/s) and air mass flow (kg/s) together define the air-fuel ratio the engine is actually running at, which governs combustion efficiency, EGT, and power output. A drifting fuel-to-air ratio without a corresponding pilot input change usually points to an injector problem.

Torque and Power

Torque (Nm) is the rotational force the engine produces; Power (W) is the rate of doing work, related by $\text{Power} = \text{Torque} \times \text{Angular Velocity} (\omega)$, where $\omega = \text{RPM} \times 2\pi / 60$. These describe how much useful mechanical output the engine is delivering for the fuel and air it is consuming — a proxy for overall engine efficiency and mechanical health.

Battery Voltage, Alternator Current & Alternator Health

The electrical subsystem that powers ignition, sensors, and avionics. A UAV cannot lose engine ignition or flight-critical sensors mid-mission, so electrical health is tracked alongside pure mechanical health.

Injection Timing

The precise crank angle (in degrees) at which fuel is injected into the cylinder relative to the piston position. Small drifts in injection timing change combustion efficiency and EGT/CHT profiles even when every other input looks normal — this is why 'injector_degradation' is tracked as its own distinct fault class rather than folded into 'overheating.'

Environmental Parameters: Altitude, Ambient Temperature, Atmospheric Pressure, Air Density

These are not engine faults themselves, but they change what 'normal' looks like. Air gets thinner (lower density) at altitude, which reduces the oxygen available for combustion and changes expected CHT/EGT/power for the same throttle input. This is precisely why the system computes physics-informed 'expected' values conditioned on altitude and ambient temperature, instead of using one fixed threshold for every flight condition.

3.3 The Key Mathematical Relationships Used in the System

3.3.1 Physics Residuals — the central idea of the whole framework

Instead of asking 'is CHT above 150°C?' (a fixed threshold — exactly what DRDO said is outdated), the system asks 'is CHT higher than physically expected given the current RPM, throttle, load, and ambient temperature?' This is a residual:

$$\Delta CHT = CHT_{\text{measured}} - CHT_{\text{expected}}$$

$$\Delta EGT = EGT_{\text{measured}} - EGT_{\text{expected}}$$

$$\Delta RPM = RPM_{\text{measured}} - RPM_{\text{expected}}$$

The 'expected' value comes from a baseline physics/thermodynamic reference model — essentially a learned or calibrated mapping from (throttle, load, ambient conditions) to the temperature/RPM a healthy engine would produce at that operating point. A residual near zero means the engine is behaving exactly as physics predicts — nominal. A residual that grows over time, even while the raw sensor value stays within a 'safe' absolute range, is early evidence of degradation. This is the physics-informed AI DRDO explicitly asked for in the innovation list.

3.3.2 Dimensionless Thermodynamic Ratios

$$\text{Fuel-to-Air Ratio: } \Phi = \dot{m}_{\text{fuel}} / (\dot{m}_{\text{air}} + \epsilon)$$

This tells you how rich or lean the combustion mixture is, independent of how much total air is flowing (which changes with altitude and throttle). The small ϵ (epsilon) in the denominator is a numerical safety constant added

purely to prevent a division-by-zero crash if air flow momentarily reads as zero — a standard engineering guard, not a physical term.

Thermal / Specific Energy Efficiency: $\eta = P_{\text{mech}} / (\dot{m}_{\text{fuel}} + \varepsilon)$

How much mechanical power the engine is extracting per unit of fuel burned — a direct efficiency signal. A quietly falling efficiency trend, even with no single alarming sensor reading, is a classic sign of gradual mechanical degradation (e.g., increasing internal friction or a slowly clogging injector).

Specific Torque Efficiency: $\tau_{\text{rpm}} = \text{Torque} / (\text{RPM} + \varepsilon)$

Normalizes torque output by engine speed, letting the model compare mechanical output fairly across different throttle settings.

3.3.3 Time-Series Behaviour: Differencing, Rolling Statistics, and Lags

A single instantaneous sensor reading cannot show a trend — trends only exist across time. The system therefore keeps a rolling buffer of the last 120 telemetry frames and computes:

- Differencing (rate of change): $\text{diff}_k(x)_t = x_t - x_{t-k}$, computed for $k = 1, 5$, and 10 steps — this is a discrete approximation of a time derivative, dx/dt , and is what lets the models notice a value that is changing quickly even if it hasn't yet crossed any absolute limit.
- Rolling mean and rolling standard deviation over windows of $5, 15$, and 30 frames — the mean smooths out sensor noise to reveal the underlying trend, and the standard deviation quantifies how erratic/unstable a signal has become (rising vibration or EGT variance is itself a fault signature, independent of the mean value).
- Lag features ($t-1, t-3, t-6, t-12$ steps) — giving the model explicit access to recent history so it can learn patterns that unfold over multiple frames, not just the current instant.
- Slope over a window: $\text{slope}_w(x) = (x_t - x_{t-w}) / \text{elapsed_hours}$ — an explicit estimate of how fast a parameter is degrading per flight-hour, which feeds directly into the Remaining Useful Life model.

3.3.4 Remaining Useful Life (RUL) and Uncertainty

RUL asks: 'given the current degradation trajectory, how many more flight-hours can this engine safely operate before it needs maintenance or fails?' Raw model output is never trusted directly in a safety context — it is passed through several physically-motivated correction layers, explained fully in Section 5.4. The key ideas worth understanding here:

- Exponential Moving Average (EMA) smoothing: $\text{filtered}_t = \alpha \times \text{raw}_t + (1 - \alpha) \times \text{filtered}_{t-1}$, with $\alpha = 0.12$ — a low-pass filter that removes high-frequency sensor jitter without lagging the real trend too much.
- Slew-rate limiting: caps how fast the displayed RUL number is allowed to jump per tick (max $+0.5\text{h}$ up, -2.0h down) — this reflects the physical reality that an engine cannot instantly regain life, but it can legitimately fail fast, so the limits are asymmetric on purpose.
- Confidence Interval (P10–P90): using the standard normal z-value 1.645 for a 90% interval, $\text{P10} = \text{RUL} - 1.645\sigma$ and $\text{P90} = \text{RUL} + 1.645\sigma$, where σ is estimated from the variance across sub-ensembles of the trained model's boosting rounds. This gives an honest 'the true RUL is probably between X and Y hours' rather than a false-precision single number.

The single sentence that ties this whole section together: every feature in this system exists to answer one physics question — 'is this engine behaving the way a healthy engine, at this exact RPM/throttle/altitude, should behave — and if not, how fast is that gap growing?' That is the definition of a physics-informed digital twin, and it is the

reason DRDO's problem statement explicitly separates 'conventional threshold-based monitoring' (bad) from 'physics-informed, AI-driven prediction' (what they want).

4. Our Approach — How We Turned the Problem Statement Into a System

The team's design philosophy was to treat the problem statement's six sub-systems (A–F) as a literal specification and build one clean software component per requirement, rather than one monolithic script. This produced a 5-tier microservices architecture where each service has a single, clearly named responsibility, communicating over REST and WebSockets — the same architectural style used in real production defense and aerospace ground systems, which strengthens the story that this is engineered, not hacked together.

The build order followed the natural data pipeline: first get real telemetry data flowing (CAN layer), then turn raw numbers into physically meaningful features (feature engine), then train models on those features (4 ML models), then make those models explainable (XAI layer), then wrap everything in real microservices with security, and finally add the two 'desired innovation' proofs of concept — Edge AI benchmarking and Federated Learning — once the core system was stable.

4.1 High-Level System Summary

Layer	What it does	Problem statement item satisfied
CAN Telemetry Layer	Encodes/decodes real engine signals over a DBC-defined CAN protocol, simulated over a virtual bus	CAN bus/SocketCAN-based data acquisition
Physics-Informed Feature Engine	Converts raw telemetry into residuals, ratios, rolling stats, lags (13 to 120 features per model)	A. Digital Twin Core Framework, Physics-informed AI
4 AI/ML Models	Anomaly detection, degradation estimation, fault classification, RUL prediction	B, C, D — Health Monitoring, Fault Detection, AI/ML Layer
Explainable AI Engine	SHAP + counterfactual sensitivity turned into plain-English maintenance advisories	Explainable AI for fault diagnosis (innovation)
Simulation & Replay Engine	Mission playback, seek, speed control, synthetic fault injection	E. Simulation & Replay Capability
5 Microservices + API Gateway	Decoupled, scalable services behind one REST/WebSocket gateway	A. Modular architecture for future scalability
MongoDB Atlas Persistence	Stores every frame, mission summary, and advisory for later analysis	Post-flight analysis, mission-wise health reports
5-Layer Security Model	TLS, service auth, rate limiting, CAN checksums, model integrity hashing	Secure telemetry architecture (innovation)
Edge AI Benchmark	Measures real inference latency to decide onboard vs. ground-station deployment	Edge AI for UAV applications (innovation)
Federated Learning PoC	FedAvg simulation across a 4-UAV fleet, quantified accuracy/privacy tradeoff	Federated learning approaches (innovation)

5. The AI/ML Layer — Every Model, Every Feature, Explained

The system runs four independent machine learning models on every telemetry frame, each answering a different diagnostic question. All four are 'frozen inference engines' — they are trained once offline and never retrained during a live mission, which guarantees deterministic, repeatable predictions during flight (a hard safety requirement, since a model that changes its own behaviour mid-flight is unauditably and unpredictable).

5.1 Model 1 — Anomaly Detection

Property	Value
Algorithm	Isolation Forest (unsupervised) + StandardScaler
Input	13 features
Output	anomaly_score, is_anomaly (bool), decision_function
Question it answers	Does this exact combination of sensor values look statistically unusual, even if no single value is individually out of range?

Why Isolation Forest, and why these 13 features

Isolation Forest is unsupervised — it does not need labelled fault examples to work. It builds random decision trees and measures how few splits it takes to isolate a data point: outliers get isolated in very few splits (short average path length across the forest), normal points take many splits. This makes it ideal for catching genuinely novel or unlabelled anomalies that were never seen during fault-model training — exactly the scenario a real UAV might face in the field.

The 13 input features are the instantaneous engine and physics-residual values (CHT, EGT, oil temperature, oil pressure, vibration RMS, battery voltage, alternator current, alternator health, injection timing, plus the CHT/EGT/RPM physics residuals and physics_residual_C). These were chosen because they are the most physically diagnostic instantaneous readings — the ones a human maintenance engineer would look at first — rather than the full rolling-feature set, since anomaly detection is meant to catch sudden, in-the-moment deviations rather than slow trends (that job belongs to the degradation model).

Output interpretation

anomaly_score is the negative of the model's decision_function (higher = more anomalous). is_anomaly is true when anomaly_score crosses a configurable threshold (default 0.0). Because it is unsupervised, this model has no accuracy/precision number in the traditional sense — its value is demonstrated live by injecting a synthetic fault and watching is_anomaly flip to true within one telemetry tick.

5.2 Model 2 — Degradation Estimation

Property	Value
Algorithm	XGBoost Regressor
Input	120 features
Output	degradation_index (0.0–1.0), estimated_health_pct (0–100%)
Question it answers	How worn is this engine right now, on a continuous scale, not just a fault yes/no?

Why XGBoost, and why 120 features

XGBoost (Extreme Gradient Boosting) builds an ensemble of decision trees sequentially, where each new tree is trained to correct the residual errors of all previous trees combined. It handles non-linear relationships between many correlated sensor signals extremely well, trains fast, and — critically for this project — is directly compatible with TreeSHAP, giving exact, fast explainability (Section 6).

The 120 features are generated by the feature engine from a rolling 120-frame history buffer: for each of 10 core signals (RPM, EGT, CHT, oil temperature, oil pressure, vibration RMS, fuel flow, air mass flow, RPM residual, EGT residual), the system computes 1/5/10-step differences, and 5/15/30-frame rolling mean and rolling standard deviation. Degradation is fundamentally a slow trend, not an instantaneous reading, so this model deliberately needs the widest feature set of all four models — it is the only one built almost entirely from trend and volatility signals rather than raw values.

Health percentage conversion

$$estimated_health_pct = (1.0 - degradation_index) \times 100$$

This is the number shown on the dashboard as engine health — a single, intuitive 0–100% figure a non-technical operator can read at a glance, backed by a continuous regression rather than a discrete threshold.

5.3 Model 3 — Fault Classification

Property	Value
Algorithm	Multiclass XGBoost Classifier + LabelEncoder
Input	55 features
Output	predicted_fault, confidence, fault_probabilities (per class)
Fault classes	normal, misfire, overheating, lubrication_degradation, injector_degradation, sensor_fault

Why multiclass classification, and why these features

Once the degradation model has said 'something is wrong,' the natural next question is 'what specifically is wrong?' — that is a classification problem, not a regression problem, so a separate model is used. The 55 input features are rolling means, rolling standard deviations, and slopes over a 15-frame window for 9 signals most diagnostic of specific fault mechanisms: CHT, EGT, oil temperature, oil pressure, fuel flow, vibration RMS, battery voltage, injection timing, and the physics residual. This is a smaller, more targeted feature set than the degradation model's 120, because fault classification needs to react faster (a misfire needs to be caught within seconds, not over many flight-hours) — so the rolling window is shorter (15 frames vs. 30) and the slope terms are explicit rather than implicit.

How each fault class maps to physics

- misfire — sudden EGT/RPM instability from a failed combustion cycle
- overheating — sustained positive CHT/EGT residual, cooling system or excessive load
- lubrication_degradation — falling oil pressure combined with rising oil temperature
- injector_degradation — drifting fuel-air ratio and injection timing signatures without a corresponding pilot input
- sensor_fault — a reading that is physically implausible given all other correlated signals (the model's way of flagging 'trust this sensor less', not an engine problem)
- normal — none of the above; all residuals within the expected envelope

5.4 Model 4 — Remaining Useful Life (RUL) Prediction

Property	Value
Algorithm	XGBoost Regressor + deterministic post-processing pipeline
Input	60 raw model features (131 after full post-processing feature expansion)
Output	predicted_rul_hours, raw_rul_hours, rul_lower_bound_p10, rul_upper_bound_p90, uncertainty_std_hours, confidence_level
Minimum history required	13 frames (returns status COLLECTING_HISTORY before that)

Why RUL needs more than a raw model prediction

This is the most safety-critical model in the system, and it is the one place where the team deliberately did not trust the raw XGBoost output directly — a raw regression prediction can jitter frame-to-frame due to sensor noise, which would be dangerous if shown directly to a pilot ('RUL: 40h... 12h... 38h...' one second apart is not useful, it's alarming and untrustworthy). Four physically-motivated correction layers sit between the raw model and the number shown on the dashboard:

- Dynamic health anchoring — blends the raw prediction with a physical target proportional to current engine health ($\text{health\%} \times 50$ hours), and hard-clamps RUL to zero once degradation reaches ≥ 0.98 (engine failure state). Formula: $\text{RUL_anchored} = 0.3 \times \text{RUL_raw} + 0.7 \times \text{RUL_target}$.
- EMA smoothing ($\alpha = 0.12$) — removes frame-to-frame sensor noise while still tracking real trend changes.
- Slew-rate limiting — caps the displayed change per tick to +0.5h rise / -2.0h drop, reflecting that RUL should recover slowly (or not at all) but can legitimately crash quickly if a real fault appears.
- Sub-ensemble uncertainty quantification — the model's own gradient-boosted trees are evaluated at 10 checkpoints along the boosting sequence (using progressively more trees each time); the variance across these checkpoint predictions estimates σ , which is then used to compute a 90% confidence interval (P10, P90) using the $z = 1.645$ standard normal multiplier.

Why 60/131 features

RUL needs the longest memory of any model, because life-remaining depends on the full degradation trajectory, not just the current or recent state. Its features include 1/3/6/12-step lags and 3/6/12-frame rolling mean/std for 9 core signals, plus explicit slope-over-elapsed-hours terms — directly encoding 'how fast is this getting worse per flight hour', which is the literal physical quantity RUL depends on.

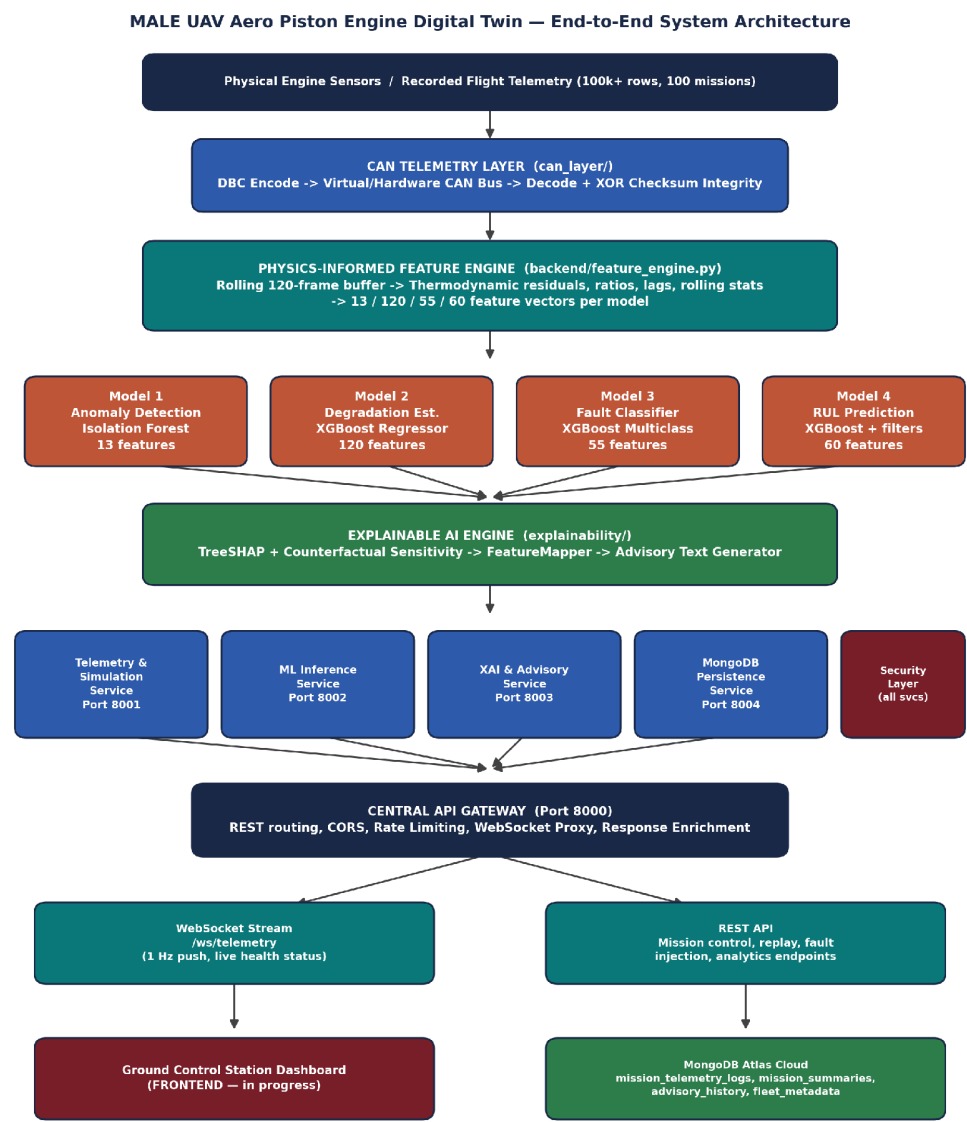
5.5 Model Comparison Summary

Model	Algorithm	# Features	Type	Latency (measured)
1. Anomaly Detection	Isolation Forest	13	Unsupervised outlier detection	21.7 ms
2. Degradation Estimation	XGBoost Regressor	120	Supervised regression (0–1)	1.07 ms
3. Fault Classification	XGBoost Multiclass	55	Supervised classification (6 classes)	2.7 ms
4. RUL Prediction	XGBoost Regressor + filters	60 (131 total)	Supervised regression + post-processing	2.7 ms

Latency numbers are real measurements taken from the actual trained model artifacts on a single CPU core — see Section 9 (Edge AI) for the full benchmarking methodology and results.

6. Complete System Architecture

The diagram below shows the full, real data path implemented in this repository, from raw engine sensors through to the (in-progress) Ground Control Station dashboard and MongoDB Atlas persistence. Every box in this diagram corresponds to a real file or service in the codebase — nothing here is aspirational.



Security Layer applied throughout: TLS/WSS, X-Internal-Key service auth, IP rate limiting, CAN frame checksums, SHA-256 model integrity hashing, secrets isolation via .env

6.1 Step-by-Step Data Flow Explanation

Step 1 — Telemetry Source

In the absence of physical ECU hardware, the system replays real recorded flight telemetry — a 100,000-row, 100-mission dataset (MALE_UAV_aero_piston_engine_final_100k.csv) plus a held-out demo mission (mission 999, used specifically for out-of-sample verification) — through `sensor_simulator.py`, which feeds rows into the pipeline exactly as a live ECU would stream them, one frame at a time.

Step 2 — CAN Telemetry Layer

Each row of telemetry is packed into real CAN 2.0B frames using a DBC (CAN database) specification (`engine_can.dbc`), grouped into 6 logical messages (engine dynamics, thermal, fuel/air, mechanical/electrical, ignition, environment). Frames are transmitted and received over a `python-can` Bus object — currently `backend="virtual"` (an in-process virtual CAN bus), with a `socketcan` backend already implemented and ready for real hardware. Every frame carries a 1-byte XOR checksum for tamper/corruption detection (Section 7.4).

Step 3 — Physics-Informed Feature Engine

`DigitalTwinFeatureEngine` maintains a rolling 120-frame buffer and, on every new frame, computes physics residuals, thermodynamic ratios, and time-series features (differences, rolling means/stds, lags), then slices out the correct feature subset for each of the four models (13 / 120 / 55 / 60 features respectively). This is the single most important transformation in the system — it is what turns raw numbers into physically meaningful signals.

Step 4 — Four Parallel Model Inferences

`DigitalTwinModelManager` runs all four frozen models against their respective feature vectors on every tick, verifying each model artifact's SHA-256 hash at load time. Outputs are combined into one status field (NOMINAL / ANOMALOUS-WARNING / FAULT DETECTED) that drives the rest of the pipeline.

Step 5 — Explainable AI Layer

`DigitalTwinXAIEngine` takes the model outputs and feature vectors and computes: TreeSHAP attributions for the three XGBoost models, and counterfactual sensitivity + z-score distance for the Isolation Forest. `FeatureMapper` translates engineered column names (e.g. `cht_C_rollmean30`) back into plain-English sensor names ('Cylinder Head Temperature'), and the advisory generator turns all of this into a human-readable maintenance recommendation.

Step 6 — Microservices & API Gateway

The five microservices (Telemetry/Simulation on 8001, ML Inference on 8002, XAI/Advisory on 8003, MongoDB Persistence on 8004, and the central API Gateway on 8000) communicate over internal REST calls authenticated with a shared X-Internal-Key header. The API Gateway is the only externally exposed service — it proxies and enriches responses from the other four, exposes a WebSocket stream at `/ws/telemetry` for live dashboard updates, and exposes REST endpoints for mission control, replay, and fault injection.

Step 7 — Persistence & Replay

Every telemetry frame, mission summary, and advisory is written to MongoDB Atlas in real time via the MongoDB Persistence Service, indexed for fast retrieval, enabling full post-flight mission replay — directly satisfying DRDO's 'post-flight analysis and mission replay' requirement.

Step 8 — Ground Control Station Dashboard (frontend, in progress)

The dashboard consumes the WebSocket stream and REST endpoints to show live health status, fault alerts, RUL trends, and mission replay — this is the visible face of the system in a demo and the one component still being finished.

7. The Security Layer — Full Explanation

Because this is a defense-context problem statement, the team implemented a genuine Defense-in-Depth security model — meaning security is layered across multiple independent controls, so that no single component failing compromises the whole system. This directly answers DRDO's 'secure telemetry architecture' innovation ask, and it is worth presenting with confidence because most competing teams are unlikely to have implemented more than a login form.

7.1 Layer 1 — Transport Layer Security (TLS / WSS)

`scripts/generate_dev_cert.py` generates a self-signed 2048-bit RSA certificate pair (`certs/server.crt`, `certs/server.key`) enabling HTTPS and secure WebSockets (`wss://`) for the dashboard connection. In a real defense deployment, self-signed certificates would be replaced with certificates issued by the defense network's own Public Key Infrastructure (PKI) — the team documents this honestly as a production upgrade path rather than pretending the dev certificate is production-grade.

7.2 Layer 2 — Service-to-Service Authentication

Only the API Gateway (port 8000) is meant to be reachable from outside. Every internal microservice (8001–8004) requires a shared-secret header, `X-Internal-Key`, validated by `verify_internal_key()` in `backend/security.py`. Any request missing or presenting the wrong key is rejected with HTTP 401 and logged as a `[SECURITY AUDIT]` event.

```
def verify_internal_key(x_internal_key: str = Header(None, alias="X-Internal-Key")):
    if not x_internal_key or x_internal_key != INTERNAL_SERVICE_KEY:
        logger.warning(f"[SECURITY AUDIT] Unauthorized inter-service request attempt.")
        raise HTTPException(status_code=401, detail="Unauthorized: Invalid or missing internal key.")
    return True
```

The planned production upgrade for this layer is mutual TLS (mTLS) using X.509 client certificates, giving every microservice a cryptographically verifiable identity instead of a single shared password — a standard step up used in zero-trust microservice meshes.

7.3 Layer 3 — API Gateway Rate Limiting

The gateway enforces a sliding-window rate limit of 60 requests per minute per client IP on the highest-risk endpoints (`/api/simulation/step` and `/api/simulation/inject_fault`), returning HTTP 429 Too Many Requests when exceeded. This is implemented as an in-memory `SlidingWindowRateLimiter` that tracks a rolling list of request timestamps per IP and discards anything older than the 60-second window.

Why this matters here specifically: fault injection is a powerful capability (it can override live sensor readings), so it is exactly the kind of endpoint that needs abuse protection even in a demo/prototype system.

7.4 Layer 4 — CAN Frame Message Integrity

Every CAN frame gets a 1-byte checksum appended before transmission, computed by XOR-ing the arbitration ID bytes with every payload byte. On decode, the checksum is recomputed and compared; a mismatch means the frame was corrupted or tampered with, and it is dropped with a `[SECURITY AUDIT]` log line instead of being silently trusted.

```
def calculate_checksum(arbitration_id: int, data_bytes: bytes) -> int:
    checksum = arbitration_id & 0xFF
    checksum ^= (arbitration_id >> 8) & 0xFF
    for b in data_bytes:
        checksum ^= b
    return checksum & 0xFF
```

```
# On decode:
if received_checksum != expected_checksum:
    logger.warning(f"[SECURITY AUDIT] CAN Frame integrity check failed... frame dropped.")
    raise ValueError("CAN Frame Integrity Checksum Mismatch")
```

The team is explicit that this XOR checksum is a demonstration-level integrity pattern, not a cryptographic authentication code — the honest production path is CAN-FD SECURE under ISO 21434 / AUTOSAR SecOC (which uses real message authentication codes/MACs), or a bus-level Intrusion Detection System watching arbitration timing patterns.

7.5 Layer 5 — Secrets Hygiene & Environment Isolation

- .env and certs/ are explicitly listed in .gitignore — never committed to the repository
- Zero hardcoded credentials — the MongoDB connection string and the internal service key are both read from environment variables via python-dotenv
- .env.example ships a safe template with placeholder values only

7.6 Layer 6 — Model Integrity & Traceability (SHA-256 Fingerprinting)

At startup, DigitalTwinModelManager computes a SHA-256 hash of every one of the four trained model files and compares it against a known-good hash stored in models/model_hashes.json. A mismatch triggers a CRITICAL [SECURITY WARNING] log — meaning if a model file were ever swapped, corrupted, or tampered with, the system would refuse to silently trust it. Every prediction response also carries the verified hashes in its metadata field, giving full auditability back to the exact verified model version that produced any given prediction.

7.7 What Is Explicitly Out of Scope (and why saying so is a strength)

The team's own security documentation explicitly lists what is not yet implemented, rather than overclaiming: user authentication / RBAC (JWT/OAuth2 operator roles), full mTLS between all internal services, Hardware Security Modules (HSM/TPM) for key storage, automated CAN-FD SecOC, and a full CAN bus Intrusion Detection System. Naming these openly, with a clear roadmap, reads as far more credible to a technical judge than silence — it shows the team understands the difference between a hackathon prototype and a fielded defense system.

8. File-by-File Codebase Walkthrough

This section goes through every significant file in the repository, what it is responsible for, and the core logic inside it. Use this as the reference if a judge asks 'walk me through your code' or 'show me where X happens.'

8.1 `can_layer/` — CAN Telemetry Protocol Layer

`can_layer/dbc.py`

Defines the DBC (CAN database) specification: 6 CAN messages, each with a fixed set of signals, their bit positions, scale factors, and offsets — `engine_can.dbc` mirrors real automotive/aerospace CAN database conventions. This is the single source of truth for how a physical value like 'CHT in °C' gets packed into raw CAN bytes.

`can_layer/can_codec.py`

Contains `encode_frame()` and `decode_frame()`: `encode_frame()` takes a physical value, applies $(\text{value} - \text{offset}) / \text{scale}$ to get a raw integer, and packs it into the correct bytes of an 8-byte CAN payload. `decode_frame()` reverses this and additionally verifies the XOR checksum described in Section 7.4, raising `ValueError` and logging a security audit event on any mismatch.

`can_layer/bus.py`

`create_bus(backend="virtual"|"socketcan", channel)` — a thin factory that returns a python-can Bus object. Currently defaults to virtual (in-process, no hardware needed); switching to real hardware later is a one-line config change to `backend="socketcan", channel="can0"`, not a rewrite.

`can_layer/sensor_simulator.py`

Reads the recorded flight CSV row by row and yields it as if it were live telemetry, at a configurable playback rate — this is what stands in for a physical ECU stream during development and demo.

`backend/can_adapter.py`

`CANTelemetryAdapter.transmit_and_receive()` is the glue: takes a telemetry dict, calls `encode_telemetry()` (which loops over `dbc.py`'s message definitions and calls `can_codec.encode_frame()` for each), sends every resulting frame on the bus, immediately receives it back, and decodes it via `decode_frame()` — producing a round-tripped, checksum-verified telemetry dict identical in shape to the original.

8.2 `backend/` — Core Digital Twin Engine

`backend/config.py`

Central configuration: model file paths, feature list definitions per model, environment variable loading (via `python-dotenv`) for the internal service key and MongoDB URI, and constants like the rolling buffer size (120 frames).

`backend/feature_engine.py` — `DigitalTwinFeatureEngine` class

The physics-informed feature factory described in detail in Section 3 and 5. Internally maintains a `collections.deque` rolling buffer of the last 120 raw telemetry frames. Public methods: `add_frame(telemetry_dict)` pushes a new frame and returns the computed feature sets; `get_features_for_model(model_name)` slices out the correct subset of columns per model (13/120/55/60); `compute_physics_residuals()` and `compute_ratios()` implement the formulas from Section 3.3.

```
def compute_physics_residuals(self, row):
    row["cht_residual"] = row["cht_C"] - row["expected_cht_C"]
```

```
row["egt_residual"] = row["egt_C"] - row["expected_egt_C"]
row["rpm_residual"] = row["rpm"] - row["expected_rpm"]
return row
```

backend/model_loader.py — DigitalTwinModelManager class

Loads all four trained model artifacts (.pkl / .json) once at service startup, verifies each against models/model_hashes.json using SHA-256 (Section 7.6), and exposes one method per model: predict_anomaly(features), predict_degradation(features), predict_fault(features), predict_rul(features, current_health). Also contains the full RUL post-processing pipeline (EMA smoothing, slew-rate limiting, sub-ensemble uncertainty — Section 5.4) and a combined predict_all() method that runs all four models and merges their outputs into one status object.

```
def _verify_model_hash(self, model_path, expected_hash):
    sha256 = hashlib.sha256()
    with open(model_path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b''):
            sha256.update(chunk)
    actual_hash = sha256.hexdigest()
    if actual_hash != expected_hash:
        logger.critical(f"[SECURITY WARNING] Model hash mismatch for {model_path}!")
    return actual_hash == expected_hash
```

backend/simulation_engine.py — MissionSimulationEngine class

The orchestrator that ties everything together for a running mission. Responsibilities: load a mission CSV, step through it frame-by-frame at a configurable playback speed, support seek (jump to any timestamp for replay), support synthetic fault injection (override specific sensor values to simulate a fault on demand — used heavily in demos), maintain alert-level state with anti-spam logic (so the same advisory isn't repeated every single tick), and generate maintenance advisory text and Return-to-Base (RTB) triggers when engine health crosses critical thresholds.

```
if health_pct < 40 and alert_state != "CRITICAL":
    msg = f"ADVISORY: Engine degradation elevated (Health Index: {health_pct:.1f}%). Schedule depot maintenance post-mission."
    if health_pct < 20:
        msg = "CRITICAL: RTB (Return to Base) recommended immediately."
    alert_state = "CRITICAL"
```

backend/security.py

Implements verify_internal_key() (Section 7.2) and SlidingWindowRateLimiter (Section 7.3) — both are FastAPI dependency functions, meaning any endpoint can opt in to either check with a single line: dependencies=[Depends(verify_internal_key)].

backend/verify_microservices.py

A standalone diagnostic script that pings every microservice's /health endpoint in sequence and prints a pass/fail report — used to quickly confirm all 5 services are up and correctly authenticated before a demo run.

8.3 explainability/ — Explainable AI Engine

explainability/shap_explainer.py

Wraps shap.TreeExplainer around each of the three XGBoost models, producing exact per-feature SHAP (SHapley Additive exPlanations) attribution values — how much each input feature pushed the prediction up or down from the model's baseline expected value. TreeExplainer is used specifically because it computes exact SHAP values for tree ensembles in polynomial time, versus the much slower model-agnostic KernelSHAP.

explainability/anomaly_explainer.py

Since Isolation Forest is not tree-based in the SHAP-compatible sense used here, this module instead computes counterfactual sensitivity — perturbing each input feature slightly and measuring how much the anomaly score changes — plus a z-score distance of each feature from its expected distribution, giving an interpretable 'which sensor is most responsible for this anomaly' answer without needing a supported SHAP backend.

explainability/feature_mapper.py — FeatureMapper class

A lookup dictionary translating engineered/internal column names (e.g. `cht_C_rollmean30`, `egt_residual_diff5`) into plain-English sensor descriptions (e.g. 'Cylinder Head Temperature — 30-frame rolling average'). Exists purely so the advisory text shown to a non-technical operator never contains a raw Python column name.

explainability/confidence.py

Utility functions for turning raw model confidence/probability outputs into calibrated, human-readable confidence bands (e.g. High / Medium / Low) shown alongside every prediction.

explainability/xai_engine.py — DigitalTwinXAIEngine class

The top-level orchestrator for this whole module: given a model's prediction and its feature vector, it calls the right explainer (SHAP for the 3 XGBoost models, counterfactual for Isolation Forest), maps features to plain English via FeatureMapper, and assembles a final advisory object combining the numeric prediction, the top contributing features, and a generated maintenance recommendation string.

8.4 services/ — The Five Microservices

Each service is an independent FastAPI application. All internal endpoints (everything except the API Gateway's public-facing routes) require the X-Internal-Key header described in Section 7.2.

services/telemetry_service/main.py — Port 8001

Owns the MissionSimulationEngine instance and CAN adapter. Endpoints: /missions (list available missions), /load_mission, /start, /pause, /step (advance one frame and return raw+decoded telemetry), /speed (change playback rate), /seek (jump to a timestamp), /inject_fault, /clear_faults, /simulate/scenario (environmental scenario simulation).

services/ml_inference_service/main.py — Port 8002

Owns the DigitalTwinModelManager and DigitalTwinFeatureEngine instances. Its single core endpoint, /predict_all, takes a telemetry frame, runs it through the feature engine, then all four models, and returns the combined prediction object. /reset_state clears the rolling buffer between missions so history from one mission never leaks into another's degradation/RUL trend.

services/xai_service/main.py — Port 8003

Owns the DigitalTwinXAIEngine. Its /explain endpoint takes a prediction + feature vector and returns SHAP/counterfactual attributions plus the generated advisory text.

services/mongodb_service/main.py — Port 8004

The only service that talks to the database (MongoDB Atlas, via pymongo). Endpoints: /log_telemetry (append a frame to mission_telemetry_logs), /log_summary, /log_advisory, /mission_replay/{mission_id} (fetch a full recorded mission for replay), /list_saved_missions, /get_advisories, /get_fleet_metadata.

services/api_gateway/main.py — Port 8000 (the only public-facing service)

Proxies and enriches every request to/from the four backend services, exposes CORS for the frontend, applies the sliding-window rate limiter (Section 7.3) on simulation/inject_fault routes, and exposes /ws/telemetry — a WebSocket endpoint that pushes a fully-assembled telemetry+prediction+advisory object to any connected dashboard once per second while a mission is running. It also exposes read-through analytics endpoints: /api/analytics/edge_benchmark and /api/analytics/federated_learning, which serve the results from Sections 9 and 10 directly to the frontend so those results can be shown live in the dashboard, not just in a static doc.

8.5 End-to-End Internal Flow — Putting It All Together

- 1. Frontend calls POST /api/simulation/step on the Gateway (port 8000).
- 2. Gateway forwards to Telemetry Service (8001), which advances the mission simulator by one frame and runs it through the CAN adapter (encode → virtual bus → decode + checksum verify).
- 3. Gateway takes the decoded telemetry and calls ML Inference Service (8002) /predict_all — which runs the Feature Engine, then all 4 models, and returns a combined prediction.
- 4. Gateway calls XAI Service (8003) /explain with the prediction, gets back SHAP attributions and advisory text.
- 5. Gateway calls MongoDB Service (8004) /log_telemetry to persist the frame + prediction + advisory.
- 6. Gateway assembles all of the above into one JSON object and pushes it down the open /ws/telemetry WebSocket connection to the dashboard, once per second while running.
- 7. If any step's internal call fails auth (wrong/missing X-Internal-Key) or the CAN checksum fails, the failure is logged as a [SECURITY AUDIT] event and surfaces as an error rather than a silently wrong prediction.

9. Edge AI Benchmark — Onboard vs. Ground Station Deployment

DRDO lists 'Edge AI for UAV applications' and 'lightweight onboard analytics' as desired innovation areas. Rather than assert this, the team built scripts/benchmark_edge_ai.py, which loads the four real trained model artifacts (the exact same files used in production inference, verified against their SHA-256 hashes) and measures real single-core CPU inference latency and on-disk file size for each.

Method

Each model is run against a representative feature vector and timed; file size on disk is used as a proxy for memory footprint. Two deployment tiers are defined by latency budget: Tier 1 (Onboard Flight Computer) requires under 2ms per inference, since a real-time RTB safety decision cannot tolerate delay; Tier 2 (Ground Control Station) allows up to 50ms, appropriate for deeper analytics running off the aircraft.

Model	Size	Latency	Deployment Tier
Anomaly Detection (Isolation Forest)	5.7 MB	21.7 ms	Ground Station
Degradation Estimation (XGBoost)	9.4 MB	1.07 ms	Onboard Flight Computer
Fault Classification (XGBoost)	1.7 MB	2.7 ms	Ground Station
RUL Prediction (XGBoost + filters)	9.0 MB	2.7 ms	Ground Station
All 4 combined	25.8 MB	28.2 ms	Near real-time (GCS)

Conclusion and talking point

Only the degradation model is fast enough to safely run onboard for instantaneous RTB triggering; the other three remain on the ground station where a 20–30ms budget is acceptable. This is a measured engineering decision, not a guess — worth stating explicitly to judges as: 'We benchmarked real inference latency on our trained artifacts and split deployment accordingly, rather than assuming everything could run onboard.'

Honesty checkpoint: file size is used as a proxy for memory footprint, not a live RAM profile (e.g. via psutil/tracemalloc). If asked directly whether RAM usage was measured, say so plainly — it strengthens credibility rather than weakening it.

10. Federated Learning Proof-of-Concept — Fleet-Wide Privacy-Preserving Training

The second desired innovation area implemented is federated learning, in scripts/federated_learning_poc.py. It answers a specific defense-relevant question: can a fleet of UAVs get a smarter shared model without any UAV ever transmitting its raw engine telemetry off the aircraft?

Method — real FedAvg (Federated Averaging), not just a description

- 1. The 100,000-row dataset is split into 4 simulated UAV nodes (UAV-01 to UAV-04) by mission ID, so each node only ever sees the missions 'it' flew — 25,000 rows each.
- 2. A local Ridge regression model (predicting engine health %) is trained independently on each node's own data only.
- 3. The 4 local models' learned weights are averaged together (the actual FedAvg algorithm) into one shared global model — no raw data from any node is ever pooled or transmitted, only the trained coefficients.
- 4. Three paradigms are compared head-to-head on the same held-out test data: Centralized (all data pooled — best possible accuracy, but requires full raw-data sharing), Local-only (each UAV alone — full privacy, weakest accuracy), and Federated (the proposed approach).

Real measured results

Setup	RMSE (lower is better)	R ²	Privacy
Centralized	2.215	0.929	Low — raw data pooled
Local-only (per UAV)	4.737	—	High — but weak model
Federated (FedAvg)	3.501	0.822	High — and much stronger than local-only

Conclusion and talking point

Federated learning closes roughly 26% of the accuracy gap between a lone UAV's local model and the ideal (but insecure) centralized model — while never transmitting a single row of raw flight telemetry off any aircraft. Say to judges: 'We didn't just claim privacy-preserving fleet learning — we quantified the exact accuracy cost of preserving that privacy, and it is a strongly favourable tradeoff for a defense fleet.'

Honesty checkpoint: the PoC uses a linear Ridge regression as a stand-in to cleanly demonstrate the FedAvg weight-averaging mechanism, not the production XGBoost models — tree ensembles cannot be weight-averaged the same way (they need federated boosting, a harder distributed-systems problem). If asked, say exactly this: it's future work, correctly scoped for a hackathon PoC.

11. Additional Recommendations (Beyond What Was Asked)

A few things worth doing before the final presentation that will materially strengthen how the project is perceived, even in the time remaining:

- Add a short 'Known Limitations' slide/section presented proactively, not waiting to be asked — covering: virtual CAN bus (not physical hardware), Ridge-regression FL proxy, file-size-as-memory-proxy for Edge AI, and no user auth/RBAC yet. Proactively naming limitations reads as engineering maturity, not weakness.
- If time allows, add a one-paragraph 'Deployment Roadmap' verbal summary drawn from docs/DEPLOYMENT_ROADMAP.docx — judges often ask 'what happens after the hackathon,' and having this ready on the tip of your tongue is a strong differentiator.
- Prepare one specific, rehearsed answer for 'why microservices for a hackathon prototype instead of one script?' — answer: it directly satisfies DRDO's explicit 'modular architecture for future scalability' requirement, and it means each of DRDO's 5 sub-systems (A–F) is independently deployable, testable, and swappable with real hardware later without touching the others.
- Run backend/verify_microservices.py fifteen minutes before you go on stage — confirm all 5 services report healthy before the judges see the dashboard, so a live demo failure doesn't happen mid-presentation.
- Keep a fallback: a short pre-recorded screen capture of the working backend (WebSocket stream, fault injection, RUL updating live) in case live Wi-Fi/network conditions fail during the actual demo slot — common at hackathons, and judges respect a team that planned for it.

12. How to Present This in the Demo

The goal of the demo is to prove, in under 5–7 minutes typically given, that this is a real working system solving DRDO's actual requirements — not a slide deck. Structure the demo around the problem statement's own six sub-systems (A–F), since that is exactly how a judge who has read the problem statement will be mentally scoring you.

Suggested demo script (5–7 minutes)

5. 0:00–0:45 — Open with the problem, not the solution. One sentence: 'DRDO's current UAV engine monitoring is threshold-based and reactive — it tells you after something has already failed. We built a physics-informed digital twin that predicts failure before it happens.' This immediately signals you understood the ask.
6. 0:45–1:30 — Show the live dashboard with a mission running: RPM/CHT/EGT/oil pressure updating in real time over the WebSocket stream, health index and RUL both visible. State: 'This is real telemetry, encoded and decoded over an actual CAN protocol layer with a DBC specification, not a mocked JSON feed.'
7. 1:30–2:30 — Trigger a live fault injection (e.g., misfire or overheating) via the `/inject_fault` endpoint on stage. Show: the anomaly score spike within one tick, the fault classifier correctly naming the fault type with a confidence score, and — critically — the SHAP-based explanation showing which sensors drove that call. This is the single most impressive live moment; do not skip it.
8. 2:30–3:15 — Show the RUL panel and explain in one sentence why it's smoothed and bounded (P10–P90), not a raw jittery number: 'We don't show pilots a number that changes every second — we show a physically-grounded, uncertainty-bounded estimate.'
9. 3:15–4:00 — Show mission replay: pick a past mission from MongoDB and replay it, proving the post-flight analysis requirement is real and working end-to-end, not just live-only.
10. 4:00–4:45 — One slide, two numbers: the Edge AI latency table and the Federated Learning accuracy/privacy table. Say the two rehearsed one-line conclusions from Sections 9 and 10. Do not over-explain the math live — the numbers speak for themselves.
11. 4:45–5:15 — One slide on security: name the 5 layers in one breath (TLS, service-key auth, rate limiting, CAN checksums, SHA-256 model integrity) and say explicitly what is NOT yet done (auth/RBAC, mTLS, HSM) — this proactive honesty is a differentiator, not a weakness to hide.
12. 5:15–5:45 — Close on scope and roadmap: 'This maps to all six of DRDO's required sub-systems: A through E are functionally complete; F, the visualization dashboard, is what you're looking at right now, finished today.'

Roles — who should say what

- Whoever is most comfortable with the ML models should own the fault-injection + SHAP explanation moment — it's the highest-impact 60 seconds of the demo.
- Whoever built the CAN/security layer should own the security slide — specificity (naming the actual checksum algorithm, the actual key header name) reads far more credible than a generic 'we made it secure.'
- Every team member should be able to answer 'why physics residuals instead of raw thresholds?' in one sentence (Section 3.3.1) — assume any judge can ask any person on the team, not just whoever is presenting.

13. Winning Strategies

13.1 Lead with what's rare, not what's common

Almost every competing team will have some kind of anomaly detection model. Very few will have all of: physics-informed residual features, TreeSHAP explainability, a real measured Edge AI latency study, a real FedAvg proof-of-concept, and a 5-layer named security model. Spend disproportionate demo time on these — they are your genuine differentiators, not the basic 'we predict faults' claim every team will make.

13.2 Answer the question the problem statement actually asks, explicitly

Judges scoring against a DRDO-issued rubric will likely check off the six A–F sub-systems almost literally. Structure your pitch deck and verbal walkthrough around those exact six letters — it makes it trivially easy for a judge to award full marks against their own checklist, instead of making them hunt for where each requirement was met.

13.3 Turn honesty about limitations into a strength

Naming the virtual CAN bus, the Ridge-regression FL proxy, and the missing user-auth layer proactively — with a clear roadmap for each — signals engineering maturity. A judge who has seen dozens of teams overclaim will notice and reward a team that clearly understands the boundary between 'built' and 'planned.'

13.4 Make the explainability moment the emotional peak of the demo

A model that says 'FAULT: misfire, 94% confidence' is good. A model that also says 'because EGT dropped 40°C below expected and vibration RMS rose 3.2 standard deviations above baseline' is what separates a black-box prediction from something a maintenance engineer could actually act on and trust. This is your single best 20-second soundbite.

13.5 Tie every technical choice back to a defense-specific reason, not a generic ML reason

Instead of 'we used XGBoost because it's accurate,' say 'we used XGBoost because it's fast enough for onboard inference (we measured 1ms) and because it supports exact SHAP explainability, which matters for a maintenance engineer who needs to trust and act on a diagnosis, not just receive a number.' Framing every choice around mission reliability and trust — not generic ML best-practice — matches the specific values a defense evaluator is listening for.

13.6 Close with confidence about scope, not apology about the frontend

Do not open or close by apologizing that the frontend was 'finished today.' Every judge knows hackathon timelines. Instead, frame it factually and confidently: the backend — which is the harder, more novel engineering problem — has been complete and iterated on for days; the dashboard is the final, visible layer connecting a genuinely deep system to the people who need to use it.

Final note: this document is long on purpose — it exists so that no single team member is a bottleneck for answering questions. Read Sections 3, 5, 7, and 13 closely before the demo; they are the ones most likely to come up in Q&A.